

PHOENIX: Pauli-Based High-Level Optimization Engine for Instruction Execution on NISQ Devices

Zhaohui Yang*, Dawei Ding[†], Chenghong Zhu[‡], Jianxin Chen[§], Yuan Xie*

*Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong

[†]Yau Mathematical Sciences Center, Tsinghua University, Beijing 100084, China

[‡]The Hong Kong University of Science and Technology (Guangzhou), Guangdong 511453, China

[§]Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China

Abstract—Variational quantum algorithms (VQA) based on Hamiltonian simulation represent a specialized class of quantum programs well-suited for near-term quantum computing applications due to their modest resource requirements in terms of qubits and circuit depth. Unlike the conventional single-qubit (1Q) and two-qubit (2Q) gate sequence representation, Hamiltonian simulation programs are essentially composed of disciplined subroutines known as Pauli exponentiations (Pauli strings with coefficients) that are variably arranged. To capitalize on these distinct program features, this study introduces PHOENIX, a highly effective compilation framework that primarily operates at the high-level Pauli-based intermediate representation (IR) for generic Hamiltonian simulation programs. PHOENIX exploits global program optimization opportunities to the greatest extent, compared to existing SOTA methods despite some of them also utilizing similar IRs. PHOENIX employs the binary symplectic form (BSF) to formally describe Pauli strings and reformulates IR synthesis as reducing the column weights of BSF by appropriate Clifford transformations. It comes with a heuristic BSF simplification algorithm that iteratively searches for the most appropriate 2Q Clifford operators to maximally simplify the BSF at each step, until the BSF can be directly synthesized by basic 1Q and 2Q gates. PHOENIX further performs a global ordering strategy in a Tetris-like fashion for these simplified IR groups, carefully balancing optimization opportunities for gate cancellation, minimizing circuit depth, and managing qubit routing overhead. Experimental results demonstrate that PHOENIX outperforms SOTA VQA compilers across diverse program categories, backend ISAs, and hardware topologies.

I. INTRODUCTION

Quantum computing offers the potential to revolutionize various fields, driving decades of efforts to develop the required physical hardware. For instance, quantum algorithms can achieve exponential speedups in tasks such as integer factorization [1], solving linear equations [2], and quantum system simulation [3]. In the noisy intermediate-scale quantum (NISQ) era where we have access to dozens or hundreds of qubits susceptible to noise (e.g., qubit decoherence, gate imperfections) [4], variational quantum algorithms (VQA) are a leading class of algorithms proposed to achieve quantum advantage (e.g., VQE for chemistry and condensed-matter simulations [5], QAOA for combinatorial optimization [6]) due to its modest resource requirements in terms of qubit number and circuit depth as well as its noise-resilience [7].

The construction of a VQA ansatz circuit aims to simulate (approximate) a desired unitary evolution under the system Hamiltonian H and the evolution duration t , through Trotterizing [8] the evolution $U(t)$ given H represented by a linear combination of Pauli strings:

$$U(t) = e^{-iHt} \simeq (S_k(\tau))^r, \quad \tau = \frac{t}{r}, \quad (1)$$

$$H = \sum_{j=1}^L h_j P_j = \sum_{j=1}^L h_j \sigma_0^{(j)} \otimes \cdots \otimes \sigma_{n-1}^{(j)}, \quad (2)$$

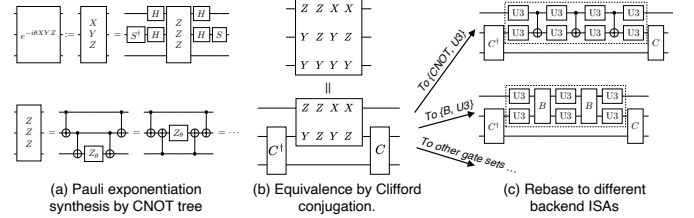


Fig. 1. Conventional Pauli exponentiation synthesis v.s. Simultaneous simplification by Clifford conjugation. (a) Pauli exponentiations are synthesized by gate set $\{H, S, S^\dagger, Z(\theta), \text{CNOT}\}$, through variable CNOT-tree unrolling schemes. (b) Multiple 3Q Pauli exponentiations can be simultaneously simplified into 2Q Pauli exponentiations, through a 2Q Clifford conjugation, where $C = (H \otimes S) \text{CNOT} (H \otimes S^\dagger)$ in this example. (c) The simplified Pauli exponentiations can be rebased to versatile quantum gate sets, such as CNOT-based and B-based ISAs, with significantly reduced 2Q gate count.

where k and r indicate the Trotter order and time step, respectively. Finer-grained Trotterization results in lower approximation errors. For example, the 1st-order and 2nd-order Trotterization are given by

$$S_1 = \prod_{j=1}^L e^{-ih_j \tau P_j}, \quad S_2 = \prod_{j=1}^L e^{-ih_j \frac{\tau}{2} P_j} \prod_{j=L}^1 e^{-ih_j \frac{\tau}{2} P_j},$$

respectively. Following Trotterization, each term S_k is expressed as a product of individual Pauli exponentiations, which can be easily synthesized using basic 1Q and 2Q gates. The arrangement of these Pauli exponentiations within each Trotter step can be freely chosen without affecting the upper bound of Trotterization (approximation) error [8], which is usually not comparable to physical noise. However, different arrangements offer varying opportunities for optimizing quantum circuits, thereby dominating the accuracy of Hamiltonian simulations on noisy hardware. Consequently, the primary challenge of compiling VQA ansatz circuits lies in synthesizing these Pauli exponentiations from a global perspective, dubbed *Pauli-based intermediate representations (IR)* throughout this paper.

Conventionally, a Pauli exponentiation is synthesized into a 1Q rotation $Z(\theta)$ sandwiched by a pair of symmetric CNOT trees, subsequently conjugated by some H and S gates, as exemplified in Fig. 1 (a). Existing state-of-the-art (SOTA) compilers primarily exploit gate cancellation opportunities exposed by the already synthesized subcircuits, whether by means of the abstract ZX diagram [9], [10], [11] representation or the variants of CNOT trees [12], [13]. Despite utilizing the Pauli-based IR, the optimization process they formulate is limited to subcircuits and local IR patterns. Furthermore, it assumes the conventional CNOT-based quantum instruction set architecture (ISA). We instead present that a set of Pauli exponentiations *can be simultaneously simplified* through appropriate Clifford

[§]Corresponding author. Email: chenjianxin@tsinghua.edu.cn.

transformations. For example, Fig. 1 (b) shows that the list of weight-3 Pauli strings $[ZYY; ZZY; XYY; XZY]$ can be simplified into a weight-2 Pauli string list through the conjugation of a 2Q Clifford operator $C = (H \otimes S) \text{CNOT} (H \otimes S^\dagger)$. This approach unlocks greater optimization opportunities *entirely at the level of Pauli-based IR* while remaining agnostic to the quantum ISA being employed.

In this work, we propose PHOENIX (*Pauli-based High-level Optimization ENgine for Instruction eXecution*)—a highly effective compilation framework for generic Hamiltonian simulation programs on near-term quantum devices. PHOENIX follows the

“IR grouping $\rightarrow$ group-wise simplification $\rightarrow$ IR group ordering”

pipeline to compile real-world VQA programs into basic quantum gates. It is ISA-independent, routing-aware, and tailored to generic VQA programs such as molecular simulation involving heterogeneous-weight Pauli strings and QAOA including only weight-2 Pauli strings. Differing from existing SOTA methods, PHOENIX utilizes an alternative formal description for Pauli-based IR and integrates heuristic optimization strategies to achieve global optimization primarily at the high-level semantic layer. We evaluate PHOENIX across diverse VQA programs, backend ISAs, and hardware topologies, demonstrating its superior performance over existing SOTA compilers. Our main contributions are as follows:

- 1) We utilize the *binary symplectic form* (BSF) as the formal description of Pauli-based IR and reformulate the IR synthesis process as simultaneous simplification on BSF by appropriate Clifford conjugations that simultaneously reduce the BSF’s column weights, enabling global optimization to the largest extent while operating in the high-level IR.
- 2) We propose a heuristic *BSF simplification* algorithm as the core optimization pass in PHOENIX. It iteratively searches for the most appropriate 2Q Clifford operators to quickly lower the weight of the BSF, until the BSF can be directly synthesized by basic 1Q and 2Q gates.
- 3) We quantify the circuit depth overhead induced by assembling (ordering) simplified IR groups. A uniform cost function is devised to comprehensively account for circuit depth overhead, gate cancellation, and qubit routing overhead. Under the guidance of that, PHOENIX integrates a *Tetris-like IR group ordering* procedure to reliably achieve a good ordering scheme.

Overall, PHOENIX outperforms the best-known compilers (e.g., TKET [14], PAULIHEDRAL [12], TETRIS [13]), achieving significant reductions in gate count and circuit depth. For example, for logical-level compilation, PHOENIX results in 80.47% reduction in CNOT gate count and 82.7% reduction in 2Q circuit depth on average, compared to the original logical circuits. For hardware-aware compilation with heavy-hex topology, PHOENIX reduces by 36.17% (22.62%) in CNOT gate count and 43.85% (28.12%) in 2Q circuit depth on average, compared to PAULIHEDRAL (TETRIS). The reduction effect in 2Q gate count and circuit depth becomes even more impressive when targeting the newly introduced SU(4) ISA (the representative continuous ISA containing all 2Q gates) [15].

II. RELATED WORK

Based on ZX diagram representations of Pauli gadgets. Compilers such as TKET [14], [9], PAULIOPT [10], and PCOAST [11] resynthesize quantum circuits into its ZX diagram representation. ZX diagrams are used to visually represent and simplify the commutation relations and phase interactions between Pauli operators, facilitating the reduction of circuit depth and gate count through algebraic and graphical transformations. Although it can efficiently resynthesize

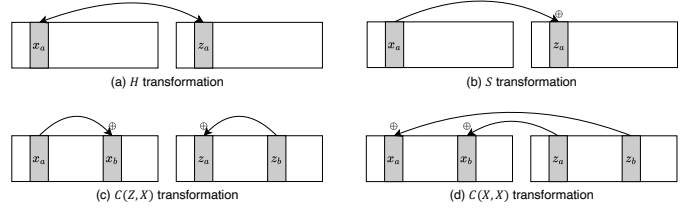


Fig. 2. Examples of Clifford transformations on BSF. Columns a and b , corresponding to qubits a and b , for X and Z blocks are indicated by x_a, x_b, z_a, z_b , respectively. (a) H gate acting on qubit a will exchange x_a and z_a . (b) S gate acting on qubit a results in $z_a \leftarrow z_a \oplus x_a$. (c) $C(Z, X)_{a,b}$, i.e., the CNOT gate, results in $x_b \leftarrow x_b \oplus x_a$ and $z_a \leftarrow z_a \oplus z_b$. (d) $C(X, X)$ transformation is equivalent to successively applying H on a , $C(Z, X)_{a,b}$, and H on a , resulting in $x_a \leftarrow x_a \oplus z_b$ and $x_b \leftarrow x_b \oplus z_a$.

Pauli gadgets, the commutation rules it leverages are local, and it is hard to operate in a hardware-aware manner.

Based on synthesis variants of Pauli-based IR. Dedicated compilers like PAULIHEDRAL [12] and TETRIS [13] leverage Pauli-based IR to identify gate cancellation opportunities between nearest-neighbor IRs, exposed by the variants of their synthesis schemes based on CNOT-tree unrolling. They proposed sophisticated co-optimization techniques that minimize the CNOT gate for both logical-level synthesis and SWAP-based routing achieve good performance especially on limited-topology NISQ devices. However, their optimization scope is confined to a finite set of local subcircuit patterns, and they rely solely on the CNOT-based ISA.

III. BSF AND CLIFFORD FORMALISM

We establish our synthesis scheme by representing Pauli strings in the binary symplectic form (BSF). In the BSF, each n -qubit Pauli string is represented as a row in a tableau, with columns divided into two sections: $[X | Z]$. Here, $[X_{i,j} | Z_{i,j}]$ denotes the j -th component of the i -th Pauli operator. The encoding maps X to $[1 | 0]$, Z to $[0 | 1]$, Y to $[1 | 1]$, and I to $[0 | 0]$. For instance, the Pauli string simplification shown in Fig. 1 (b) follows the formulation

$$\left[\begin{array}{cc|cc} 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 1 \end{array} \right] \xrightarrow{C(X,Y)_{1,2}} \left[\begin{array}{cc|cc} 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \end{array} \right],$$

where $C(X, Y)_{1,2}$ is the Clifford gate in Fig. 1 (b) while written in the *universal controlled gate* representation

$$C(\sigma_0, \sigma_1) = \frac{1}{2}((I + \sigma_0) \otimes I + (I - \sigma_0) \otimes \sigma_1), \sigma_j \in \{X, Y, Z\}.$$

Any universal controlled gate is equivalent to CNOT up to local H and S conjugations. For example, $C(Z, X) = \text{CNOT}$, and $C(X, Y) = (H \otimes S) \text{CNOT} (H \otimes S^\dagger)$.

By definition, any operator C from the Clifford group is unitary and has the property that $CPC^\dagger$ is also a Pauli operator for any Pauli operator P . The update rules of some Clifford operators in terms of the BSF are demonstrated in Fig. 2, in which only 2Q Clifford operators (Clifford2Q) potentially have nontrivial effects of reducing the Pauli strings’ weights. For any $C(\sigma_0, \sigma_1)$, its tableau update rule is a combinatorial sequence of H , S and $C(Z, X)$ (or $C(Z, Y)$) transformations. For example, $C(X, Y)_{a,b}$ exhibits the rule

$$[x_a, x_b | z_a, z_b] \rightarrow [x_a \oplus x_b \oplus z_b, z_a \oplus z_b | z_a, z_a \oplus z_b]. \quad (3)$$

Therefore, the basic approach of our synthesis scheme is to find an appropriate sequence of Clifford2Q operators to simplify the BSF¹ until it can be directly synthesized by basic 1Q and 2Q gates, i.e., the quantity “total weight”

$$w_{\text{tot.}} := \|\bigvee_i (r_x^{(i)} \vee r_z^{(i)})\| \quad (4)$$

is no more than 2, where $r_{x/z}^{(i)}$ is the i -th row of X or Z block of the BSF, and the norm is the sum of all binary entries. The key lies in how to find the Clifford2Q that can reduce the weights of as many Pauli strings as possible within the BSF.

IV. OUR PROPOSAL: PHOENIX

A. Overview

In the proposed framework PHOENIX, Pauli-based IRs (exponentiations) are first grouped according to the same set of qubit indices non-trivially acted on. Then, the *BSF simplification* algorithm is applied to the BSF of each IR group, generating the simplified subcircuit composed of CNOT-equivalent Clifford2Q operators and Pauli exponentiations with weights no more than 2. PHOENIX further selectively assembles (orders) these unarranged simplified IR groups in a Tetris-like style that minimizes a uniform subcircuit assembling cost function that incorporates gate cancellation, circuit depth increase, and qubit routing overhead. By defining suitable metric functions and heuristic algorithms, PHOENIX achieves superior optimization (fewer 2Q gates and reduced circuit depth) compared to SOTA methods, efficiently handling compilation for large-scale VQA programs.

B. BSF simplification for each IR group

To search for appropriate Clifford2Q operators in BSF simplification, it suffices to just focus on a set of Clifford2Q generators of the 2Q Clifford group. We choose the six universal controlled gates

$$\{C(X, X), C(Y, Y), C(Z, Z), C(X, Y), C(Y, Z), C(Z, X)\} \quad (5)$$

that are independent of each other and span the entire 2Q Clifford group [16]. It is a natural choice of Clifford2Q generators, as each of them is Hermitian and equivalent to CNOT, and its corresponding BSF’s tableau update rule can be defined as a combination of the update rules for H , S , and CNOT.

To determine which Clifford2Q operator to select from (5), we define a heuristic cost function

$$\begin{aligned} \text{cost}_{\text{bsf}} := & w_{\text{tot.}} * n_{\text{n.l.}}^2 + \sum_{\langle i, j \rangle} \|r_x^{(i)} \vee r_z^{(i)} \vee r_x^{(j)} \vee r_z^{(j)}\| \\ & + \frac{1}{2} \sum_{\langle i, j \rangle} (\|r_x^{(i)} \vee r_x^{(j)}\| + \|r_z^{(i)} \vee r_z^{(j)}\|) \end{aligned} \quad (6)$$

to quantify the disparity of the current BSF from a desired BSF that requires no further simplification ($w_{\text{tot.}} \leq 2$). cost_{bsf} is the combined weight overlap of both X -part and Z -part among each pair of Pauli strings of a BSF, with a bias considering the impact of the number of nonlocal (n.l., that is, weight larger than 1) Pauli strings.

Algorithm 1 illustrates the BSF simplification procedure in detail. It takes set of Pauli strings as input and outputs a configuration sequence cfg , in which each component is either a Clifford2Q generator from (5) or a BSF with $w_{\text{tot.}}$ at most 2. Specifically, at each Clifford2Q search epoch, Crefalgo:simplification identifies the Clifford 2Q generator from (5) and qubit pair to act upon that leads to the greatest reduction in the cost function defined in (6), and the BSF is updated accordingly. This greedy process continues until the BSF’s $w_{\text{tot.}}$ is no more than 2. Before each search epoch, local Pauli

Algorithm 1: Pauli Strings Simplification in BSF

Input : Pauli strings list pls
Output: Reconfigured circuit components list cfg

```

1  $\text{cfg} \leftarrow \emptyset$ ;  $\text{bsf} \leftarrow \text{BSF}(\text{pls})$ ;  $\text{cliffs\_with\_locals} \leftarrow \emptyset$ ;
2 while  $\text{bsf.TOTALWEIGHT}() > 2$  do
3    $\text{local\_bsf} \leftarrow \text{bsf.POPLOCALPAULIS}()$ ;
4    $C \leftarrow \emptyset$ ; // Clifford2Q candidates
5    $B \leftarrow \emptyset$ ; // Each element of B results from applying each
   Clifford2Q candidate on bsf
6    $\text{costs} \leftarrow \emptyset$ ; // Cost functions calculated on each element of B
7   for  $\text{cg}$  in CLIFFORD_2Q_SET do
8     for  $i, j$  in COMBINATIONS(RANGE( $n$ ), 2) do
9        $\text{cliff} \leftarrow \text{cg.ON}(i, j)$ ; // qubits acted on
10       $\text{bsf}' \leftarrow \text{bsf.APPLYCLIFFORD2Q}(\text{cliff})$ ;
11       $\text{cost} \leftarrow \text{CALCULATEBSFCOST}(\text{bsf}')$ ;
12       $C.\text{APPEND}(\text{cliff})$ ;
13       $B.\text{APPEND}(\text{bsf}')$ ;
14       $\text{costs}.\text{APPEND}(\text{cost})$ ;
15    end
16  end
17   $\text{bsf} \leftarrow \text{BSFWITHMINCOST}(B, \text{costs})$ ;
18   $\text{cliff} \leftarrow \text{CLIFFORDWITHMINCOST}(C, \text{costs})$ ;
19   $\text{cliffs\_with\_locals}.\text{APPEND}((\text{cliff}, \text{local\_bsf}))$ ;
20 end
21  $\text{cfg}.\text{APPEND}(\text{bsf})$ ;
22 for  $\text{cliff}, \text{local\_bsf}$  in  $\text{cliffs\_with\_locals}$  do
   // Clifford2Q operators are added as conjugations, with local
   Pauli strings peeled before each epoch
23    $\text{cfg}.\text{PREPEND}(\text{cliff})$ ;
24    $\text{cfg}.\text{APPEND}(\text{local\_bsf})$ ;
25    $\text{cfg}.\text{APPEND}(\text{cliff})$ ;
26 end

```

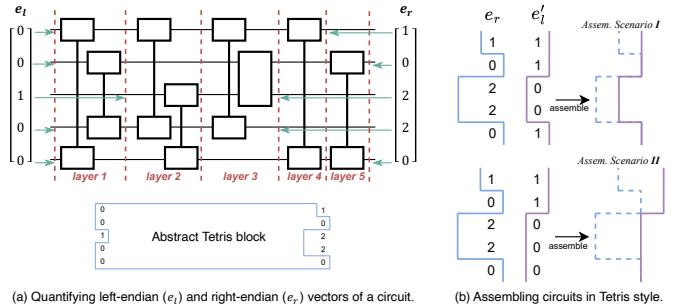


Fig. 3. Tetris-like circuits assembling. (a) e_l and e_r examples. (b) Scenario I: $\text{cost}_{\text{depth}} = \text{SUM}(e_r + e_l' - 1)$; Scenario II: $\text{cost}_{\text{depth}} = \text{SUM}(e_r + e_l')$.

strings are peeled from the BSF, as they represent 1Q Pauli rotations and do not induce synthesis overhead. The output cfg represents a simplified IR group, while still in high-level semantics (Clifford2Q, 1Q Pauli-based IRs, and successive 2Q Pauli-based IRs), independent of any specific quantum ISA.

In theory, simplifying a BSF is always achievable, for example, we can reduce an individual Pauli string to be weight-1, peel it off, and then iterate. The efficacy of Algorithm 1 lies in its simultaneous simplification mechanism guided by the cost function in (6).

C. Ordering IR groups in a Tetris-like style

With simplified IR groups compiled via BSF simplification, PHOENIX further performs a *Tetris-like IR group ordering* procedure that aims to minimize a *uniform assembling cost metric*. This is unlike

¹The BSF that needs to simplify does not contain any weight-1 Pauli string.

[12] and [13] where the ordering is primarily determined by gate cancellation opportunities between IR groups. We can abstract the circuit structure exhibited by each simplified IR group into something resembling a Tetris block. Then, we choose the ordering based on the circuit-depth cost function of assembling Tetris blocks, with gate cancellation opportunities and qubit routing overhead also taken into account. We primarily have three ingredients:

- 1) *Endian vectors of circuits and the depth cost function.* As shown in Fig. 3 (a), we define a pair of vectors e_l and e_r for each subcircuit (which would correspond to a simplified IR group in our case). The i -th entry of e_l (e_r) refers to how many layers one has to traverse starting from the left (right) side before qubit i is acted upon. The layers are defined by grouping neighboring 2Q gates that act on different qubits. See Fig. 3 (a) for an example of a generic (not necessarily simplified IR group) circuit. With this definition, the depth overhead for assembling two subcircuits—the proceeding subcircuit with left-endian vectors e_r and the succeeding subcircuit with left-endian vector e'_l is given by

$$\text{cost}_{\text{depth}} := \begin{cases} \text{SUM}(e_r + e'_l - 1), & \text{if } \text{ALL}(e_r[e'_l == 0] > 0) \\ & \text{and } \text{ALL}(e'_l[e_r == 0] > 0) \\ \text{SUM}(e_r + e'_l), & \text{otherwise} \end{cases}$$

See Fig. 3 (b) for an example.

- 2) *Clifford2Q cancellation.* The simplified IR group usually exposes Hermitian Clifford2Q operators at the two ends of the subcircuit. Therefore some Clifford2Q gate cancellation opportunities could be exploited, which may or may not decrease overall circuit depth, as depicted in Fig. 4 (a). We can add this consideration into $\text{cost}_{\text{depth}}$: if m pairs Clifford2Q cancelled, while (a) without decreasing subcircuit depth, $\text{cost}_{\text{depth}} \leftarrow \text{cost}_{\text{depth}} - 2m$; (b) with one side's subcircuit depth decreases, $\text{cost}_{\text{depth}} \leftarrow \text{cost}_{\text{depth}} - 2m - n$; (c) with both sides' subcircuit depths decrease, $\text{cost}_{\text{depth}} \leftarrow \text{cost}_{\text{depth}} - 2m - 2n$.
- 3) *Consideration of qubit routing overhead.* We try to mitigate the routing overhead for hardware-aware compilation by comparing the qubit interaction graphs of subcircuits. This approach is not limited to specific routing schemes (e.g. three-CNOT unrolling of SWAP-based routing, ancilla-based bridge gate [17]) and hardware topologies. Intuitively, two subcircuits with more “similar” qubit interaction behaviors require less mapping transition overhead between them, as demonstrated by Fig. 4 (b). Therefore, we introduce a factor characterized by the similarity between the qubit interaction graphs of two subcircuits in the cost function:

$$\text{cost}_{\text{depth}} \leftarrow \frac{1}{s} \cdot \text{cost}_{\text{depth}}, \quad s = \sum_i \frac{\langle D_i, D'_i \rangle}{\|D_i\|_2 \|D'_i\|_2}, \quad (7)$$

where D (D') is the distance matrix of the preceding (succeeding) subcircuit's qubit interaction graph of the tail (head) part. The tail (head) is defined as starting from the right (left) of the subcircuit and incorporating more and more 2Q gates until all qubits are acted upon. $D_{i,j}$ represents the shortest path length between qubit i and qubit j . D_i is the i -th row of D .

In practice, these IR groups are first pre-arranged in descending order of their weight (subcircuit width). PHOENIX looks ahead for a certain number of subcircuits to find the one with the minimum $\text{cost}_{\text{depth}}$ with respect to the last assembled subcircuit. This process iterates until all subcircuits are assembled.

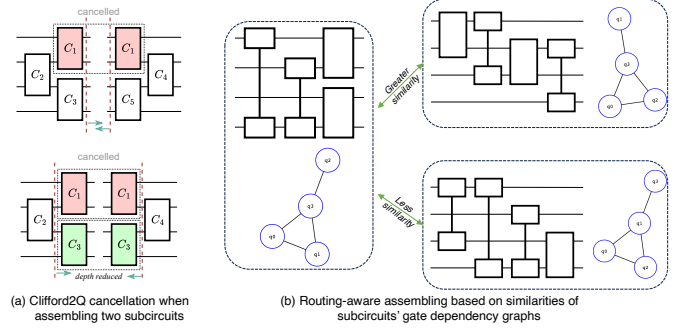


Fig. 4. Gate cancellation opportunities and routing-aware assembling. (a) Clifford2Q cancellation between the preceding and succeeding subcircuits may or may not induce circuit depth decrease. (b) The subcircuit (upper right) whose qubit interaction graph is more similar to that of the already assembled subcircuit (left) is preferred over the other (lower right).

TABLE I
UCCSD BENCHMARK SUITE.

Benchmark	#Qubit	#Pauli	w_{max}	#Gate	#CNOT	Depth	Depth-2Q
CH2_cmplt_BK	14	1488	10	37780	19574	23568	19399
CH2_cmplt_JW	14	1488	14	34280	21072	23700	19749
CH2_frz_BK	12	828	10	19880	10228	12559	10174
CH2_frz_JW	12	828	12	17658	10344	11914	9706
H2O_cmplt_BK	14	1000	10	25238	13108	15797	12976
H2O_cmplt_JW	14	1000	14	23210	14360	16264	13576
H2O_frz_BK	12	640	10	15624	8004	9691	7934
H2O_frz_JW	12	640	12	13704	8064	9332	7613
LiH_cmplt_BK	12	640	10	16762	8680	10509	8637
LiH_cmplt_JW	12	640	12	13700	8064	9342	7616
LiH_frz_BK	10	144	9	2890	1442	1868	1438
LiH_frz_JW	10	144	10	2850	1616	1985	1576
NH_cmplt_BK	12	640	10	15624	8004	9691	7934
NH_cmplt_JW	12	640	12	13704	8064	9332	7613
NH_frz_BK	10	360	9	8303	4178	5214	4160
NH_frz_JW	10	360	10	7046	3896	4640	3674

V. EVALUATION

We evaluate the effectiveness of PHOENIX across diverse Hamiltonian simulation programs, quantum ISAs, and device topologies. Although PHOENIX is implemented in Python, it compiles VQA programs of thousands of Pauli strings and more than ten qubits (approximately corresponding to the program size with 10^4 - 10^6 CNOT gates in conventional synthesis) within a few dozen seconds. Code and data are available on GitHub [18]. All experiments are executed on a laptop (Apple M3 Max, 36GB memory).

A. Experimental settings

Metrics. We evaluate PHOENIX using the following metrics: 2Q gate count, 2Q circuit depth, and the algorithmic error for VQA synthesis. Algorithmic error refers to the deviation between the synthesized circuit's unitary matrix and the ideal evolution under the original Hamiltonian, as measured by the infidelity between unitary matrices in our evaluation: $\text{infid} = 1 - \frac{1}{N} |\text{Tr}(U^\dagger V)|$. Notably, we exclude 1Q gates and their count from circuit depth measurements, as 1Q gates are generally considered free resources due to their significantly lower error rates. Additionally, CNOT is not a native operation on most NISQ platforms, requiring extra 1Q drives before and/or after native 2Q gates (e.g., Cross-Resonance [19], Mølmer-Sørensen [20]), thus including 1Q gates in metrics could be misleading.

Baselines. TKET, PAULIHEDRAL, and TETRIS are primary baselines to be compared with our method. For TKET, the PauliSimp and FullPeepholeOptimise passes are adopted for logical

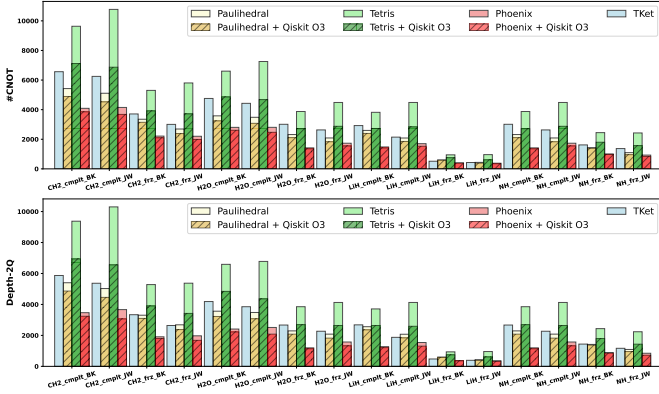


Fig. 5. Logical-level compilation (all-to-all topology).

TABLE II
AVERAGE (GEOMETRIC-MEAN) OPTIMIZATION RATES ON UCCSD.

Compiler	#CNOT opt.	Depth-2Q opt.
TKET	33.07%	30.14%
PAULIHEDRAL	28.41%	29.07%
PAULIHEDRAL + O3	25.72% (-8.54% v.s. no O3)	26.3% (-8.6% v.s. no O3)
TETRIS	53.66%	53.26%
TETRIS + O3	36.73% (-30.94% v.s. no O3)	36.37% (-31.08% v.s. no O3)
PHOENIX	21.13%	19.3%
PHOENIX + O3	19.53% (-6.6% v.s. no O3)	17.3% (-8.44% v.s. no O3)

circuit optimization. It is similar to TKET’s O3 compilation in which the `PauliSimp` pass is particularly effective at optimizing Pauli gadgets. For PAULIHEDRAL, the QISKIT O2 pass is associated by default because the numerous gate cancellation opportunities exposed by PAULIHEDRAL necessitate inverse and commutative cancellations. For hardware-aware compilation, all baselines and PHOENIX are followed by a QISKIT O3 pass with SABRE qubit mapping [21]. For QAOA benchmarking, 2QAN [22] is used as the SOTA baseline.

Benchmarks. We select two representative VQA benchmarks—(1) *UCCSD*: A set of UCCSD ansatzes, including CH_2 , H_2O , LiH , and NH : 4 categories of molecule simulation programs. Each category is generated with STO-3G orbitals [23], encoded by Jordan-Wigner (JW) [24] and Bravyi-Kitaev (BK) [25] transformations, in turn approximated by complete or frozen-core orbitals. Details are shown in Table I. (2) *QAOA*: A set of 2-local Hamiltonian simulation programs corresponding to random graphs and regular graphs, for which the description and evaluation results are shown in Table IV.

B. Main results

Fig. 5 and Table II illustrates the main benchmarking results regarding logical-level compilation:

- 1) PHOENIX significantly outperforms baselines across all benchmarks, with an average (geometric-mean) 21.12% and 19.29% optimization rate in #CNOT and Depth-2Q, respectively, relative to original circuits.² That is mostly attributed to the group-wise BSF simplification mechanism, as PHOENIX adopts the same IR grouping method as PAULIHEDRAL and TETRIS.
- 2) TETRIS performs the worst, falling far behind TKET, PAULIHEDRAL, and PHOENIX. This is because TETRIS focuses

²For example, the #CNOT optimization rate is defined as $\frac{\text{\#CNOT}_{\text{after}}}{\text{\#CNOT}_{\text{before}}}$.

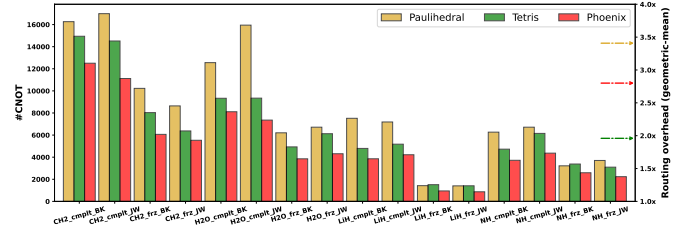


Fig. 6. Hardware-aware compilation on heavy-hex topology. Three dashed lines represent the average multiples of #CNOT within circuits after mapping relative to those after logical optimization, for PAULIHEDRAL (gold), TETRIS (green), and PHOENIX (coral), respectively.

primarily on co-optimization techniques that reduces SWAP gates during qubit routing, rather than logical-level synthesis.

- 3) We also compare PAULIHEDRAL/TETRIS/PHOENIX with and without QISKIT O3, to evaluate their high-level optimization capabilities. The improvement in using QISKIT O3 for PAULIHEDRAL and TETRIS is more pronounced than for PHOENIX. Therefore, PHOENIX’s high-level optimization strategy is more impressive, leaving less optimization space for QISKIT O3.

C. Hardware-aware compilation

We use the heavy-hex topology, specifically a 64-qubit coupling graph of IBM’s Manhattan processor [26], for hardware-aware compilation. Results are shown in Fig. 6, where TKET is excluded due to its significantly worse performance compared to others. Despite focusing primarily on high-level logical program optimization with humble hardware-aware co-optimization, PHOENIX still outperforms baselines, reducing CNOT gate count by 35.26% (21.69%) and 2Q circuit depth by 43.54% (27.68%) on average, compared to PAULIHEDRAL (TETRIS). Considering the qubit mapping transition overhead mitigation within its IR group ordering process, PHOENIX induces 2.83x #CNOT on average after hardware mapping. It is better than PAULIHEDRAL while worse than TETRIS, as TETRIS specializes in CNOT cancellation for SWAP-based routing. Consequently, even on limited-topology devices, PHOENIX effectively manages routing overhead and surpasses co-design local optimization strategies.

D. Comparison in diverse ISAs

Quantum ISA, or the native gate set in a narrow sense, serves as an interface between software and hardware implementation. For a specific quantum ISA, the adopted 2Q gate dominates the accuracy and difficulty of hardware implementation, as well as the theoretical circuit synthesis capabilities. While traditional ISAs typically consist of 1Q gates and CNOT-equivalent 2Q gates, recently some works propose integrating complex and even continuous 2Q gates into the ISA design, such as the XY gate family [27], the fractional/partial ZZ and MS gates [28], [29], and the AshN gate scheme which considers all possible 2Q gates within the $\text{SU}(4)$ group as the ISA [15]. Therefore, we further compare PHOENIX with baselines in different ISAs to showcase its ISA-independent compilation advantage.

The ISA-independent advantage is best shown by choosing the most expressive $\text{SU}(4)$ ISA. We evaluate it with both all-to-all and heavy-hex topologies, as the same as the evaluation above for CNOT ISA. For logical-level compilation, PHOENIX directly generates $\text{SU}(4)$ -based circuits via its BSF simplification algorithm, while baselines (PAULIHEDRAL and TETRIS equipped with QISKIT O3 by default) require an additional “rebase” (or “transpile”) step to convert CNOT-based circuits to $\text{SU}(4)$ -based ones. For hardware-aware compilation, all compilers include a rebase step, following the

TABLE III
COMPARISON FOR DIVERSE ISAS WITH ALL-TO-ALL AND LIMITED TOPOLOGY.

	CNOT ISA (all-to-all)		SU(4) ISA (all-to-all)		CNOT ISA (heavy-hex)		SU(4) ISA (heavy-hex)	
PHOENIX's opt. rate	#CNOT	Depth-2Q	#SU(4)	Depth-2Q	#CNOT	Depth-2Q	#SU(4)	Depth-2Q
PHOENIX v.s. TKET	63.88%	64.05%	56.06%	54.25%	41.05%	48.48%	44.65%	50.85%
PHOENIX v.s. PAULIHEDRAL	82.14%	73.4%	75.6%	65.24%	63.03%	54.7%	40.16%	35.16%
PHOENIX v.s. TETRIS	57.53%	53.08%	56.57%	50.58%	76.76%	71.43%	62.74%	58.9%

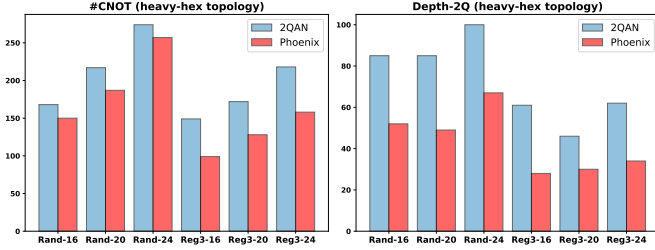


Fig. 7. QAOA benchmarking

TABLE IV
QAOA BENCHMARKING VERSUS 2QAN.

QAOA		#CNOT		Depth-2Q		#SWAP		Routing overhead	
Bench.	#Pauli	2QAN	Phoenix	2QAN	Phoenix	2QAN	Phoenix	2QAN	Phoenix
Rand-16	32	168	150	85	52	37	29	2.62x	2.34x
Rand-20	40	217	187	85	49	47	39	2.71x	2.34x
Rand-24	48	274	257	100	67	63	56	2.85x	2.68x
Reg3-16	24	149	99	61	28	44	17	3.10x	2.06x
Reg3-20	30	172	128	46	30	46	23	2.87x	2.13x
Reg3-24	36	218	158	62	34	62	30	3.03x	2.19x
Avg. improv.		-16.7%		-40.8%		-29.41%		-16.59%	

QISKIT O3 hardware-aware compilation pass. Detailed outcomes are summarized in Table III, highlighting the geometric-mean relative optimization rates of PHOENIX compared to the baselines' results.

Again, PHOENIX significantly outperforms baselines when targeting SU(4) ISA. The optimization rates relative to baselines are more impressive than those in CNOT ISA, despite the baselines incorporating sophisticated optimization techniques specifically designed for the CNOT ISA. For instance, the multiple of PHOENIX's #2Q relative to PAULIHEDRAL's in CNOT ISA is 82.12% (62.38%), whereas this value decreases to 75.57% (39.84%) in SU(4) ISA for hardware-agnostic (hardware-aware) compilation. One exception is the hardware-aware compilation comparison with TKET, as TKET's hardware-aware compilation in the CNOT ISA generates circuits with much larger 2Q gate count and circuit depth than other compilers, and there are numerous 2Q subcircuit fusing opportunities such that the rebased circuits involves much fewer SU(4) gates.

E. QAOA benchmarking

For QAOA benchmarking, we focus on the performance in hardware-aware compilation, since the 2Q gate count cannot be reduced and minimizing circuit depth is easy in logical-level compilation. Both 2QAN and PHOENIX can generate depth-optimal QAOA circuits at the logical level in our field test. Fig. 7 and Table IV illustrate compilation results on the heavy-hex topology, across six QAOA programs corresponding to both random graphs (each node with degree 4) and regular (each node with degree 3) graphs, with qubit sizes of 16, 20, and 24. PHOENIX outperforms 2QAN across all benchmarks in all metrics (such as #CNOT, #SWAP), especially in Depth-2Q, with an average 40.8% reduction compared to 2QAN. These results further demonstrate the effectiveness of the routing-aware IR group ordering method in PHOENIX.

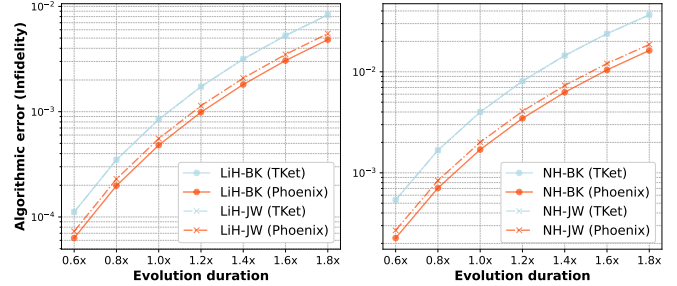


Fig. 8. Algorithmic error comparison of LiH and NH simulation.

F. Algorithmic error analysis

We further highlight PHOENIX's advantage in reducing the algorithmic error. We select UCCSD benchmarks with qubits no more than 10 for evaluation, within the computation capabilities of standard PCs. We rescale the coefficients of Pauli strings to control their algorithmic errors within 5×10^{-5} to 10^{-2} , which corresponds to different evolution durations in molecular simulation, as suggested in Fig. 8. In contrast to TKET, PHOENIX typically leads to lower algorithmic errors for both JW and BK encoding schemes. Although this improvement is program-specific, it is more pronounced for the Pauli string patterns of BK than for those of JW, with 57% (42.7%) and 49.5% (34.1%) for NH (LiH) simulation, respectively. As they adopt the same Pauli string blocking approach, we expect the algorithmic errors resulting from PAULIHEDRAL and TETRIS to be comparable to PHOENIX, and so are not shown in Fig. 8. As a result, the impressive algorithmic error reduction effect of PHOENIX brings us closer to a possible quantum advantage on computational chemistry problems.

VI. CONCLUSION AND OUTLOOK

Mainstream circuit synthesis approaches rely on pattern rewrite rules, often restricted to small-scale, local optimizations. In contrast, we present PHOENIX, a framework leveraging high-level Pauli-based IR for Hamiltonian simulation, one of the most prominent NISQ applications. PHOENIX outperforms all SOTA VQA compilers across diverse programs and hardware platforms, showcasing its unmatched performance and versatility. This work not only bridges the gap between impactful quantum applications and physically implementable solutions but also prompts a re-evaluation of compiler optimization.

Beyond producing significantly optimized circuits, high-level IRs, as a refined abstraction layer on top of the traditional quantum ISA, create more opportunities for efficient synthesis and effective hardware mapping strategies. Furthermore, they can act as intermediate building blocks, guiding the design of quantum processors and corresponding control schemes—potentially incorporating multi-qubit control—to implement these high-level IRs more efficiently.

ACKNOWLEDGEMENT

This work is supported by AI Chip Center for Emerging Smart Systems (ACCESS), sponsored by InnoHK funding, Research Grants

Council of HKSAR (No. 16213824) and National Key Research and Development Program of China (No. 2023YFA1009403), National Natural Science Foundation of China (No. 12347104), Beijing Natural Science Foundation (No. Z220002). DD would like to thank God for all of His provisions.

REFERENCES

- [1] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proceedings 35th annual symposium on foundations of computer science*. Ieee, 1994, pp. 124–134.
- [2] A. W. Harrow, A. Hassidim, and S. Lloyd, "Quantum algorithm for linear systems of equations," *Physical review letters*, vol. 103, no. 15, p. 150502, 2009.
- [3] S. Lloyd, "Universal quantum simulators," *Science*, vol. 273, no. 5278, pp. 1073–1078, 1996.
- [4] J. Preskill, "Quantum computing in the nisq era and beyond," *Quantum*, vol. 2, p. 79, 2018.
- [5] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien, "A variational eigenvalue solver on a photonic quantum processor," *Nature communications*, vol. 5, no. 1, p. 4213, 2014.
- [6] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," *arXiv preprint arXiv:1411.4028*, 2014.
- [7] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles, "Variational quantum algorithms," *Nature Reviews Physics*, vol. 3, no. 9, pp. 625–644, 2021.
- [8] A. M. Dalzell, S. McArdle, M. Berta, P. Bienias, C.-F. Chen, A. Gilyén, C. T. Hann, M. J. Kastoryano, E. T. Khabiboulline, A. Kubica *et al.*, "Quantum algorithms: A survey of applications and end-to-end complexities," *arXiv preprint arXiv:2310.03011*, 2023.
- [9] A. Cowtan, S. Dilkes, R. Duncan, W. Simmons, and S. Sivarajah, "Phase gadget synthesis for shallow circuits," *arXiv preprint arXiv:1906.01734*, 2019.
- [10] A. M. van de Griend, "Towards a generic compilation approach for quantum circuits through resynthesis," *arXiv preprint arXiv:2304.08814*, 2023.
- [11] J. Paykin, A. T. Schmitz, M. Ibrahim, X.-C. Wu, and A. Y. Matsuura, "Pcoast: a pauli-based quantum circuit optimization framework," in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 1. IEEE, 2023, pp. 715–726.
- [12] G. Li, A. Wu, Y. Shi, A. Javadi-Abhari, Y. Ding, and Y. Xie, "Paulihedral: a generalized block-wise compiler optimization framework for quantum simulation kernels," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 554–569.
- [13] Y. Jin, Z. Li, F. Hua, T. Hao, H. Zhou, Y. Huang, and E. Z. Zhang, "Tetris: A compilation framework for vqa applications in quantum computing," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 277–292.
- [14] S. Sivarajah, S. Dilkes, A. Cowtan, W. Simmons, A. Edgington, and R. Duncan, " $t|ket\rangle$: a retargetable compiler for nisq devices," *Quantum Science and Technology*, vol. 6, no. 1, p. 014003, 2020.
- [15] J. Chen, D. Ding, W. Gong, C. Huang, and Q. Ye, "One gate scheme to rule them all: Introducing a complex yet reduced instruction set for quantum computing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. La Jolla, CA, USA: ACM, 2024, pp. 779–796.
- [16] D. Grier and L. Schaeffer, "The classification of clifford gates over qubits," *Quantum*, vol. 6, p. 734, 2022.
- [17] T. Itoko, R. Raymond, T. Imamichi, and A. Matsuo, "Optimization of quantum circuit mapping using gate transformation and commutation," *Integration*, vol. 70, pp. 43–50, 2020.
- [18] "PHOENIX GitHub repo," <https://github.com/iqubit-org/phoenix>.
- [19] C. Rigetti and M. Devoret, "Fully microwave-tunable universal gates in superconducting qubits with linear couplings and fixed transition frequencies," *Physical Review B—Condensed Matter and Materials Physics*, vol. 81, no. 13, p. 134507, 2010.
- [20] C. D. Bruzewicz, J. Chiaverini, R. McConnell, and J. M. Sage, "Trapped-ion quantum computing: Progress and challenges," *Applied Physics Reviews*, vol. 6, no. 2, 2019.
- [21] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for nisq-era quantum devices," in *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*, 2019, pp. 1001–1014.
- [22] L. Lao and D. E. Browne, "2qan: A quantum compiler for 2-local qubit hamiltonian simulation algorithms," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 351–365.
- [23] W. J. Hehre, R. F. Stewart, and J. A. Pople, "Self-consistent molecular-orbital methods. i. use of gaussian expansions of slater-type atomic orbitals," *The Journal of Chemical Physics*, vol. 51, no. 6, pp. 2657–2664, 1969.
- [24] P. Jordan and E. Wigner, "über das paulische äquivalenzverbot. z phys 47: 631," 1928.
- [25] S. B. Bravyi and A. Y. Kitaev, "Fermionic quantum computation," *Annals of Physics*, vol. 298, no. 1, pp. 210–226, 2002.
- [26] G. J. Mooney, G. A. White, C. D. Hill, and L. C. Hollenberg, "Whole-device entanglement in a 65-qubit superconducting quantum computer," *Advanced Quantum Technologies*, vol. 4, no. 10, p. 2100061, 2021.
- [27] D. M. Abrams, N. Didier, B. R. Johnson, M. P. d. Silva, and C. A. Ryan, "Implementation of xy entangling gates with a single calibrated pulse," *Nature Electronics*, vol. 3, no. 12, pp. 744–750, 2020.
- [28] IBM Quantum, "New fractional gates reduce circuit depth for utility-scale workloads," <https://www.ibm.com/quantum/blog/fractional-gates>, 2024, accessed: Nov. 18, 2024.
- [29] IonQ, "Getting started with ionq's hardware-native gateset," <https://docs.ionq.com/guides/getting-started-with-native-gates>, 2023.